CONTENTS

- 1 Executive Summary
- 2 Registry Snapshot
- 3 Parameter Groups
  - 3.1 1. Trade Strength
  - 3.2 2. Confirmation Filter
  - 3.3 3. Macro / News
  - 3.4 4. Global Risk
  - 3.5 5. Gamma-Vol Acceleration
  - 3.6 6. Dealer Hedging Pressure
  - 3.7 7. Option Efficiency / Expected Move
  - 3.8 8. Strike Selection
  - 3.9 9. Event Window Policy
  - 3.10 10. Evaluation Thresholds
  - 3.11 11. Large-Move Probability
  - 3.12 12. Backtest / Harness Parameters
- 4 Recommended Tuning Methodology By Group
  - 4.1 Tune First
  - 4.2 Tune Second
  - 4.3 Leave Alone For Now
- 5 Overfitting Risk Review
  - 5.1 Highest Risk
  - 5.2 Medium Risk
  - 5.3 Lower Risk
- 6 Practical Recommendation
- 7 Appendix: Remaining Intentionally Non-Centralized Areas

1 Executive Summary

The repository now has a materially broader governed tuning surface than it did earlier in the project. The parameter registry currently covers `616` parameters across `12` logical groups:

- `trade_strength`
- `confirmation_filter`
- `macro_news`
- `global_risk`
- `gamma_vol_acceleration`
- `dealer_pressure`
- `option_efficiency`
- `evaluation_thresholds`
- `strike_selection`
- `large_move_probability`

- `event_windows`
- `keyword_category`

The most important architectural change is that the system is no longer limited to tuning only top-level thresholds and penalties. A large portion of the raw overlay feature math is now exposed through registry-governed policy configs, especially in:

- `global_risk`
- `gamma_vol_acceleration`
- `dealer_pressure`
- `option_efficiency`

This is a meaningful improvement because it brings more of the true live model surface into the same research, validation, and promotion framework.

The signal-evaluation-first principle still holds:

`market data -> signal generation -> signal evaluation dataset -> tuning / validation / promotion`

Actual executed trades remain outside the learning loop.

2 Registry Snapshot

GROUP	COUNT	MAIN SOURCES
`trade_strength`	48	`config/signal_policy.py`
`confirmation_filter`	28	`config/signal_policy.py`
`macro_news`	50	`macro/macro_news_config.py`
`global_risk`	122	`config/global_risk_policy.py`
`gamma_vol_acceleration`	85	`config/gamma_vol_acceleration_policy.py`
`dealer_pressure`	101	`config/dealer_hedging_pressure_policy.py`
`option_efficiency`	30	`config/option_efficiency_policy.py`
`evaluation_thresholds`	27	`config/signal_evaluation_scoring.py`
`strike_selection`	55	`config/strike_selection_policy.py`
`large_move_probability`	20	`config/large_move_policy.py`
`event_windows`	15	`config/event_window_policy.py`
`keyword_category`	35	`config/news_category_policy.py`

3 Parameter Groups

3.1 1. Trade Strength

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Direction vote weights	`config/signal_policy.py`, `strategy/trade_strength.py`	directional vote contribution from flow, gamma, hedging, vanna, charm	Registry-governed	Coordinate descent and small bounded search
Trade strength scoring weights	`config/signal_policy.py`, `strategy/trade_strength.py`	additive signal strength from flow, walls, gamma, liquidity, vol	Registry-governed	Group-level campaign
Consensus scoring	`config/signal_policy.py`, `strategy/trade_strength.py`	alignment bonuses and conflict penalties	Registry-governed	Coordinate descent
Runtime thresholds	`config/signal_policy.py`, `engine/trading_engine.py`	trade/watchlist/quality transitions	Registry-governed	Tune first with walk-forward validation
Direction thresholds	`config/signal_policy.py`	minimum vote score and vote margin	Registry-governed	Tune first

3.2 2. Confirmation Filter

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Confirmation thresholds and veto rules	`config/signal_policy.py`, `strategy/confirmation_filters.py`	strong/confirmed/mixed states, move-prob cutoffs, veto sensitivity	Registry-governed	Coordinate descent with walk-forward validation

3.3 3. Macro / News

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Headline classification config	`macro/macro_news_config.py` , `news/classifier.py`	hit weights, vol bonuses, bias weights	Registry-governed	Latin hypercube on bounded ranges
Aggregation config	`macro/macro_news_config.py` , `macro/macro_news_aggregator.py`	decay, confidence, headline velocity weighting	Registry-governed	Regime-aware tuning only
Regime config	`macro/macro_news_config.py` , `macro/macro_news_aggregator.py`	risk-on / risk-off mapping	Registry-governed	Walk-forward and regime-aware only
Adjustment config	`macro/macro_news_config.py` , `macro/engine_adjustments.py`	score, confirmation, and size-cap overlays	Registry-governed	High-priority tuning group
Category multipliers	`config/news_category_policy.py` , `news/classifier.py`	category-level sentiment, vol, impact, India/global bias scaling	Registry-governed	Tune category weights, not individual keywords
Raw keyword dictionaries	`news/keyword_rules.py`	actual keyword match universe	Hard-coded by design	Keep fixed for now

3.4 4. Global Risk

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Policy thresholds	`config/global_risk_policy.py`, `risk/global_risk_regime.py`	regime thresholds, overnight thresholds, size caps, adjustment scores	Registry-governed	Walk-forward and regime-aware only
Raw feature thresholds	`config/global_risk_policy.py`, `risk/global_risk_features.py`	oil, gold, copper, VIX, US equity, rates, FX shock cutoffs	Registry-governed	Bounded Latin hypercube only
Blend coefficients	`config/global_risk_policy.py`, `risk/global_risk_features.py`, `risk/global_risk_regime.py`	risk-off / risk-on pressure, global score composition, overnight gap composition	Registry-governed	Tune as grouped campaigns, not individually
Engine modifiers	`config/global_risk_policy.py`, `engine/trading_engine_support.py`	extra penalties from vol explosion and oil shock	Registry-governed	Tune conservatively after regime validation

3.5 5. Gamma-Vol Acceleration

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Regime thresholds	`config/gamma_vol_acceleration_policy.py`, `risk/gamma_vol_acceleration_regime.py`	risk-state bands, directional thresholds, overnight thresholds	Registry-governed	Tune first inside this group
Raw feature coefficients	`config/gamma_vol_acceleration_policy.py`, `risk/gamma_vol_acceleration_features.py`	gamma, flip proximity, vol transition, vacuum, hedging, macro/global blend	Registry-governed	Group-level Latin hypercube plus coordinate refinement

3.6 6. Dealer Hedging Pressure

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Regime thresholds	`config/dealer_hedging_pressure_policy.py`, `risk/dealer_hedging_pressure_regime.py`	pressure, pinning, two-sided instability, overnight thresholds	Registry-governed	Tune first inside this group
Raw feature coefficients	`config/dealer_hedging_pressure_policy.py`, `risk/dealer_hedging_pressure_features.py`	gamma base, flow confirmation, structure, vacuum, macro/global blend	Registry-governed	Group-level campaign with strict robustness filtering

3.7 7. Option Efficiency / Expected Move

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Classifier thresholds	`config/option_efficiency_policy.py`, `risk/option_efficiency_layer.py`	efficiency bands, overnight penalties, adjustment rules	Registry-governed	Tune early to mid priority
Raw feature coefficients	`config/option_efficiency_policy.py`, `risk/option_efficiency_features.py`	IV normalization, effective delta clamps, convexity multiplier, expected option move scaling	Registry-governed	Tune as grouped coefficients, not as isolated parameters

3.8 8. Strike Selection

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Strike ranking heuristics	`config/strike_selection_policy.py`, `strategy/strike_selector.py`	moneyness bands, premium bands, liquidity cutoffs, wall penalties, IV preference	Registry-governed	Group-level campaign after core signal thresholds

3.9 9. Event Window Policy

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Event window geometry and penalties	`config/event_window_policy.py` , `macro/scheduled_event_risk.py` , `engine/trading_engine.py`	warning/lockdown/cooldown windows, severity risk levels, event penalties	Registry-governed	Conservative bounded search only

3.10 10. Evaluation Thresholds

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Composite score weights	`config/signal_evaluation_scoring.py` , `research/signal_evaluation/evaluator.py`	direction / magnitude / timing / tradeability blend	Registry-governed	Research-only tuning
Selection policy thresholds	`config/signal_evaluation_scoring.py` , `tuning/objectives.py`	which signals are selected in experiments	Registry-governed	Tune carefully; high circularity risk

3.11 11. Large-Move Probability

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
Probability coefficients	`config/large_move_policy.py` , `models/large_move_probability.py`	probability uplift / penalties from gamma, vacuum, hedging, IV percentile, range	Registry-governed	Latin hypercube on narrow bounds

3.12 12. Backtest / Harness Parameters

GROUP	WHERE USED	WHAT IT CONTROLS	GOVERNANCE STATE	RECOMMENDED TUNING METHOD
TP / SL defaults	`config/settings.py`, `strategy/exit_model.py`	hypothetical trade geometry	Settings-based	Leave mostly fixed for now
Slippage / spread / commissions	`config/settings.py`, backtest modules	backtest realism assumptions	Settings-based	Do not tune for alpha
Sweep grids	`config/settings.py`, `backtest/parameter_sweep.py`	research search scaffolding	Settings-based	Adjust only as research tooling, not model tuning

4 Recommended Tuning Methodology By Group

4.1 Tune First

1. Trade strength runtime thresholds and direction thresholds
2. Confirmation filter thresholds
3. Macro/news adjustment weights
4. Global risk regime and overnight thresholds
5. Option efficiency thresholds
6. Gamma-vol and dealer-pressure regime thresholds

4.2 Tune Second

1. Global risk raw blend coefficients
2. Gamma-vol raw feature coefficients
3. Dealer-pressure raw feature coefficients
4. Option-efficiency raw feature coefficients
5. Strike-selection heuristics
6. Large-move probability coefficients

4.3 Leave Alone For Now

1. Raw keyword lists in `news/keyword_rules.py`
2. Risk-free rate and dividend yield assumptions
3. Backtest friction settings
4. Default TP / SL percentages until more outcome calibration is available

5 Overfitting Risk Review

5.1 Highest Risk

- category and overlay coefficient tuning in low-sample regimes
- evaluation-selection thresholds in `config/signal_evaluation_scoring.py`
- strike-selection heuristics in `strategy/strike_selector.py`
- any attempt to tune raw keywords instead of category multipliers

5.2 Medium Risk

- global risk and macro/news regime thresholds
- dealer-pressure and gamma-vol raw blends
- large-move probability coefficients

5.3 Lower Risk

- trade strength runtime thresholds
- direction thresholds
- confirmation thresholds

6 Practical Recommendation

The tuning program should now be thought of in three layers:

1. tune the core threshold groups first
2. tune the overlay coefficient groups only after the thresholds are stable
3. keep keyword universes and execution-external artifacts outside the tuning surface

The important improvement versus the earlier system is that the second layer is now genuinely available for governed research instead of remaining buried in feature code.

7 Appendix: Remaining Intentionally Non-Centralized Areas

- `news/keyword_rules.py`
  - `HEADLINE_RULES`
  - `POSITIVE_KEYWORDS`
  - `NEGATIVE_KEYWORDS`
- some research-harness settings in `config/settings.py`
- scenario fixture JSON files under `config/`

These remain intentionally less dynamic because they are better treated as taxonomy, research scaffolding, or operational assumptions rather than direct optimization targets.